CTCPU

CTCPU is a processor I am designing to work with two representations of a number. It uses ordinary positional notation and a second representation I call shadow. Hunter continues this work.

Start with colours

A normal processor works with bits: 0 or 1. CTCPU works with a larger smallest unit, the xbit: three bits, which means eight values. I show those eight values as eight colours, because three light channels (red, green, blue) can be mixed into eight colours. Each colour has a number and a letter.

Number	Colour	Letter	Three bits
0	Black	K	000
1	Red	R	001
2	Green	G	010
3	Yellow	Y	011
4	Blue	B	100
5	Magenta	M	101
6	Cyan	C	110
7	White	W	111

The colours mix like light: red (1) + green (2) = yellow (3), red (1) + blue (4) = magenta (5), green (2) + blue (4) = cyan (6), and red + green + blue = white (7).

A number is a row of these colours. Written in base 8, 137 is 211, so in colours it is G R R. The Mersenne number 31 is 37 in base 8, so it is Y W. The number 127 is 177 in base 8, so it is R W W. This is why the 1, 3, 7 patterns in the journal can be seen as colours.

What I want to build

My aim is a machine that wins through calmness. I want to avoid expanding a difficult number when its structure already tells me what I need. I think of it as a specialist beside the host computer, working on exact arithmetic and structured numbers. I want the emulator to meet people on the machines they already own.

That is an aim, not a measured speed or energy advantage. I keep the number, its representation and the cost of moving between representations in the same picture. Pascal patterns and Mersenne numbers brought me to many of these questions. A useful representation, a divisibility filter and a proof of primality are still different things.

Two ways to hold a number

The normal side, `rm`, holds eight octal digits, or 24 bits. For unsigned magnitude that gives 0 through 16,777,215. A colour labels a digit. It does not add information beyond its three bits.

The shadow uses the fixed primes 2, 3, 5, 7, 11, 13 and 17. For 360, the exponents are `[3,2,1,0,0,0,0]`, because `360=2^3×3^2×5`. Addition stays on the normal side. Multiplying smooth factors, which have no prime factors outside these seven primes, can become addition of their exponents.

The whole shadow is more than a list of factors. When a residual remains, I keep its last octal digit and take the rest downstairs. For 59, the first row's prime product is 1, the next row's is 7, and the saved digit is 3:

```
59 = 1 × (8 × 7 + 3)
```

Every descent divides by 8. The saved digit is in the row I descend into. I explain the filter, exponent counters and staircase in Resolver. The all-zero exponent row has value 1. It is not an empty row.

I also carry a type and a scale. A root, reciprocal, sum or factorial can remain a typed expression until I need to open it. Keeping the label does not perform the expansion. The ISA page gives the eight types and the limits of their implementation.

Parts together

The current Hunter layout has one core, one thread and eight registers, `R0..R7`. Each register contains eight rows of eight three-bit cells, or 192 bits in all:

```
rm          ordinary number
rs1..rs4    shadow rows
rc          rough cache
rpow        scale declaration
rtyp        number type and intended status storage
```

`rs1` is a row inside a register, not the register `R1`. `rc` is the rough cache. The carry from addition travels between digits and is a different object.

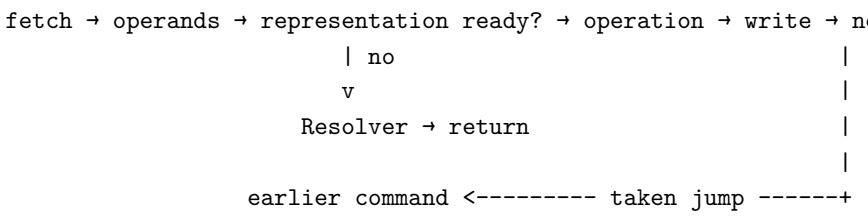
I list all the parts and their status in CTCPU parts: the cell and registers, ALU, adder, MUL_DIV, SQRT, Resolver and control functions. I also keep the older PCCPU, overflow-chain and Cube ideas there with their historical status. The former 137-bit layout, separate FLAG and VLINK do not belong to this 192-bit layout.

Execution logic: the journey of one instruction

Before execution there must be a program, input values and a starting instruction address. The program counter identifies the next instruction. Control reads its operation, modifiers

and operand selectors. The precise Hunter address and timing interface is still open, but the work has to follow this data dependency:

STEP	State before	Work	State after and decision
1	Program counter identifies a command	Fetch its four-xbit word	Instruction available to control
2	OP, MOD, A and B available	Select the operation and source registers	Operand values available
3	Operation selected	Decide whether it needs a shadow	Normal path continues, missing shadow requests Resolver
4	Shadow requested, if needed	Scan digits, count powers, process the residual	Continue only when the required representation is ready
5	Operands ready	Execute the selected arithmetic	Result and relevant conditions available
6	Result available	Write destination and maintain representation validity	New destination value committed
7	Command complete	Select sequential continuation or a jump	Program counter identifies the next command



For ADD, I read ordinary digits. For a shadow multiplication, a missing shadow triggers Resolver first. A simple smooth multiplication can add exponents, but general rough and staircase work cannot be skipped. A positional write must also maintain or invalidate the old shadow.

The program's condition or counter makes a loop end. A system stop request is intended to stop execution, but the Hunter SYS subcode and jump-target encoding remain open. I do not attach a fixed physical cycle count to these seven explanatory steps.

A small example: 6×7

I use the separate HunterCPU teaching model to show a complete small loop. It has its own 8-bit registers and instruction set. This is repeated addition, not a claim that Hunter's MUL family or a modern processor's MUL must work this way.

`R2` holds the addend 7. `R1` holds the accumulated result. `R0` counts copies still to add. The model loads R0=6 and R2=7, checks the zero case before subtracting, loads the first 7 directly from RAM into R1, then reduces R0 to 5. One copy is already present, so five additions remain.

```
R1=7, R2=7, R0=5
        |
        v
   R0 zero? ---- yes ----> store R1 → halt
        | no
        v
   R1 = R1 + R2
        |
   carry? ------ yes ----> overflow exit
        | no
        v
   R0 = R0 - 1
        |
        +---------------> return to zero check
```

STEP	R1 before	R0 before	Work	R1 after	R0 after	Next decision
Start	0	6	Load first 7, subtract one copy	7	5	Continue
1	7	5	Add 7, check carry, decrement	14	4	Continue
2	14	4	Add 7, check carry, decrement	21	3	Continue
3	21	3	Add 7, check carry, decrement	28	2	Continue
4	28	2	Add 7, check carry, decrement	35	1	Continue
5	35	1	Add 7, check carry, decrement	42	0	Finish at next zero check
End	42	0	Store result, halt	42	0	Stopped

In this model, OR checks R0 and sets Z. JZ reads Z and takes the finish branch when it is 1. ADD supplies the new sum and carry C. JC reads C for the overflow exit. With no carry, the counter decreases and JMP returns to the loop head. The equality `R1 + R0×R2 = 42` holds at every loop entry. Each completed turn moves one 7 from the remaining copies into R1. When R0 reaches zero, R1 must be 42.

The 2 October 2026 report records the following counts for that teaching program:

Counted event	Count
Logical copies of 7	6
Result additions	5
R2 reads as the result-ADD source	5
Counter decrements	6, one before the loop and five within it
Loop zero checks / JZ decisions	6, five continue and one finishes
Separate initial zero check	1
Carry checks / JC decisions	5
Loop-back JMP executions	5
All ADD instructions, including counter work	11
Completed instructions, including HLT	45
Model ticks	268
Memory reads / writes	59 / 1

R2 is loaded once, then read five times as the addend. The initial copy in R1 comes from RAM, not from an extra R2 read. These source-read counts do not count every internal transfer in the microsteps. The model's 268 ticks are a recorded emulator result, not a physical Hunter timing measurement, and I have not rerun that report here.

Within an addition, the old R1 travels through BUS into TMP. ALU combines R2 with TMP and puts the result in ACC. The next transfer takes ACC through BUS into R1. The screen's temporary purple "shadow" is this ACC stage. It is not the prime-exponent shadow `rs1..rs4`. R0 decreases later, after the carry check. In the recorded run, 42 reaches R1 at tick 222 and the program halts at tick 268.

Zero must be handled before forming `count-1`. One copy needs no repeated result addition. Negative inputs need a sign rule, and an 8-bit result beyond 255 takes the overflow path. This small unsigned example does not supply a general signed multiplication routine.

Limits I keep visible

My exact-arithmetic aim does not remove finite storage limits. The ordinary field is 24 bits, an exponent cell is three bits, the shadow has four rows, and type and scale details are not all settled. A missing terminal rule cannot be replaced by silently treating an unused row as zero or one.

Resolver's old divisibility checks are historical emulator results. Its layered rough representation is still absent from that code. General rough arithmetic, end-of-chain handling, `rc` interpretation and parts of the ISA remain open. Random inputs do not automatically have a useful smooth part. I cannot infer a speed advantage from a shorter mathematical expression.

The journal's 22 test cards check journal claims, not an integrated CPU. This package includes 30 archived tests and two programs under TESTLER/NIHAI_EK. The archive contains teaching models and starting RTL work, but this publication supplies no accepted FPGA implementation or silicon result. MHz, watts and physical cycle counts are not measured here. The old 4096-thread vision belongs to the project's history, not the current one-core capacity.

Parts, Resolver, ISA, Türkçe

James Augustus, 2026. Licensed under CC BY 4.0. https://github.com/JamesAugustus/ctcpu-hunter